

Tugas Besar 2
Convolutional Neural Network dan Recurrent Neural Network
IF3270 Pembelajaran Mesin



Joel Hotlan Haris Siahaan	13523025
Julius Arthur	13523030
Salman Hanif	13523056

Program Studi Teknik Informatika
Institut Teknologi Bandung
2026

A. Deskripsi Persoalan

Tugas Besar 2 IF3270 Pembelajaran Mesin ini berfokus pada implementasi dan eksplorasi dua arsitektur *deep learning* utama, yaitu Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN).

Bagian pertama adalah membuat arsitektur CNN menggunakan dataset Intel Image Classification yang terdiri dari kurang lebih 25.000 gambar dengan 6 kategori: *buildings*, *forest*, *glacier*, *mountain*, *sea*, dan *street*. Dataset sudah dibagi menjadi split train, validation, dan test.

Terdapat beberapa bagian utama di implementasi CNN. Pertama, implementasi utility functions untuk memuat dan memproses gambar menggunakan PIL/Pillow dan NumPy tanpa Keras preprocessing, mencakup image loader, batch loader, dan feature extractor yang menyimpan hasil ke disk dalam format .npy.

Kedua, implementasi forward propagation CNN from scratch menggunakan NumPy, yang mampu memuat bobot hasil pelatihan Keras dan mereproduksi hasilnya. Layer yang wajib diimplementasikan mencakup Conv2D dengan shared parameters, LocallyConnected2D dengan non-shared parameters, MaxPooling2D, AveragePooling2D, GlobalAveragePooling2D, GlobalMaxPooling2D, Flatten, serta fungsi aktivasi ReLU dan Softmax.

Ketiga, pelatihan dan variasi hyperparameter menggunakan Keras pada dua jenis model, yaitu Conv2D dan LocallyConnected2D, dengan variasi pada jumlah layer konvolusi (3 variasi), banyak filter per layer (3 variasi), ukuran filter (3 variasi), dan jenis pooling layer (max vs. average). Metrik utama yang digunakan adalah macro F1-score.

Keempat, evaluasi dan analisis meliputi perbandingan hasil from scratch vs. Keras, analisis pengaruh setiap hyperparameter, serta perbandingan antara model shared dan non-shared parameter dari sisi performa, jumlah parameter, dan efisiensi.

Bagian kedua adalah implementasi RNN dan LSTM menggunakan dataset Flickr8k yang terdiri dari 8.092 gambar dengan 5 caption per gambar, dibagi menjadi 6.000 data latih, 1.000 validasi, dan 1.000 data uji.

Arsitektur yang digunakan mengacu pada paper Show and Tell (Vinyals et al., 2015) dengan metode injeksi pre-inject, yaitu fitur gambar yang diekstraksi oleh CNN encoder (pretrained InceptionV3/VGG16, frozen) diproyeksikan melalui sebuah Dense, lalu dijadikan input pada timestep $t = -1$ sebelum token start.

Terdapat beberapa bagian utama di RNN/ LSTM. Pertama, implementasi forward propagation from scratch untuk layer Embedding, SimpleRNN cell, LSTM cell, Dense projection, dan Dense output, seluruhnya berbasis NumPy dan memuat bobot dari Keras. Kedua, ekstraksi fitur CNN menggunakan model pretrained dari Keras, dengan hasil disimpan ke disk dalam format .npy untuk digunakan bersama oleh kedua decoder. Ketiga, preprocessing caption meliputi tokenisasi, pembangunan vocabulary dengan token khusus start, end, dan pad, serta padding sequence. Keempat, pelatihan decoder untuk dua arsitektur (SimpleRNN dan LSTM) dengan variasi jumlah layer recurrent (1, 2, 3 layer) dan ukuran hidden state (128, 512). Kelima, evaluasi dan analisis mencakup

perbandingan BLEU-4 dan METEOR score antar variasi, perbandingan Keras vs. from scratch dari sisi skor dan waktu eksekusi, analisis kualitatif RNN vs. LSTM pada 10 contoh gambar, serta pengaruh panjang maksimum caption terhadap BLEU-4.

B. Pembahasan

1. Penjelasan Implementasi

1.1. Deskripsi kelas beserta deskripsi dan atribut dan methodnya

1.1.1. Kelas Conv2D(src/cnn/layers/conv2d.py)

Merepresentasikan convolution layer dengan parameter sharing pada CNN yang berfungsi melakukan ekstraksi fitur dari input menggunakan kernel, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Mengambil kernel, bias, aktivasi, stride, dan padding dari layer Keras.
<code>forward(self, x)</code>	Melakukan konvolusi 2D, menambahkan bias, dan menerapkan fungsi aktivasi pada output.

1.1.2. Kelas LocallyConnected2D(src/cnn/layers/locallyconnected.py)

Merepresentasikan convolution layer tanpa parameter sharing pada CNN yang berfungsi melakukan ekstraksi fitur dari input menggunakan kernel, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Mengambil kernel, bias, ukuran kernel, dan fungsi aktivasi dari layer Keras.

forward(self, x)	Melakukan operasi locally connected 2D pada input dan menerapkan fungsi aktivasi pada output. .
------------------	---

1.1.3. Kelas MaxPooling2D(src/cnn/layers/pooling2d.py)

Merepresentasikan max pooling layer pada CNN yang berfungsi melakukan downsampling dengan mengambil nilai maksimum pada setiap pool window, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Mengambil ukuran pooling dan stride dari layer Keras.
forward(self, x)	Melakukan operasi max pooling pada input.

1.1.4. Kelas AveragePooling2D(src/cnn/layers/pooling2d.py)

Merepresentasikan average pooling layer pada CNN yang berfungsi melakukan downsampling dengan mengambil nilai rata-rata pada setiap pool window, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Mengambil ukuran pooling dan stride dari layer Keras.
forward(self, x)	Melakukan operasi average pooling pada input.

1.1.5. Kelas GlobalAveragePooling2D(src/cnn/layers/pooling2d.py)

Merepresentasikan global average pooling layer pada CNN yang berfungsi melakukan downsampling dengan mengambil nilai rata-rata global input, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Menginisialisasi layer global average pooling.
<code>forward(self, x)</code>	Menghitung rata-rata global pada dimensi spasial input.

1.1.6. Kelas GlobalMaxPooling2D(src/cnn/layers/pooling2d.py)

Merepresentasikan global max pooling layer pada CNN yang berfungsi melakukan downsampling dengan mengambil nilai maksimum global input, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Menginisialisasi layer global max pooling.
<code>forward(self, x)</code>	Mengambil nilai maksimum global pada dimensi spasial input.

1.1.7. Kelas Flatten(src/cnn/layers/flatten.py)

Merepresentasikan flatten layer pada CNN yang berfungsi mengubah feature map multidimensi menjadi vektor satu dimensi, termasuk metode forward propagation pada layer tersebut.

Method:

Method	Deskripsi
<code>__init__(self, keras_layer)</code>	Menginisialisasi layer flatten.
<code>forward(self, x)</code>	Mengubah input multidimensi menjadi vektor 1 dimensi per batch.

1.1.8. Fungsi Aktivasi(src/cnn/layers/activation.py)

Merepresentasikan dan menerapkan fungsi aktivasi untuk menambah non-linearitas pada model.

Kelas:

Class	Deskripsi
ReLU	Menerapkan fungsi aktivasi ReLU pada input.
Sigmoid	Menerapkan fungsi aktivasi sigmoid pada input.
Tanh	Menerapkan fungsi aktivasi tanh pada input.
Softmax	Menerapkan fungsi aktivasi softmax pada input.
Linear	Mengembalikan input tanpa perubahan.

Method:

Method	Deskripsi
<code>__call__(self, x)</code>	Menerapkan fungsi aktivasi pada input dan mengembalikan hasilnya.

1.1.9. Kelas `EmbeddingLayer(src/rnn/layers/embedding.py)`

Merepresentasikan embedding layer yang berfungsi untuk mengkonversi token integers menjadi dense vector representations. Layer ini melakukan lookup pada embedding matrix berdasarkan input token indices, menghasilkan embedding vectors yang akan digunakan sebagai input untuk RNN/LSTM layers. Embedding layer tidak memiliki learnable parameters dalam konteks ini karena weights sudah di-freeze dari Keras training

Method :

Method	Deskripsi
<code>__init__(self, vocab_size, embed_dim)</code>	Inisialisasi layer dengan ukuran vocabulary dan dimensi embedding.
<code>load_weights(self, keras_layer)</code>	Memuat embedding matrix dari layer Keras yang sudah di-train.
<code>forward(self, x)</code>	Melakukan lookup embedding berdasarkan input indices. Input shape [batch_size, seq_len], output shape [batch_size, seq_len, embed_dim].

Contoh penggunaan: untuk batch 32 captions dengan panjang 35 tokens dan embed_dim 256, forward pass akan mengkonversi integer token sequence menjadi dense embeddings dengan shape [32, 35, 256].

1.1.10. Kelas `SimpleRNNLayer(src/rnn/layers/simple_rnn.py)`

Merepresentasikan Simple RNN cell yang melakukan sequence processing dengan recurrent connections. Layer ini mempertahankan hidden state yang diupdate pada setiap timestep berdasarkan input saat ini dan hidden state sebelumnya. SimpleRNN layer ini mengimplementasikan forward propagation dari scratch menggunakan NumPy, dengan weight matrices yang di-load dari Keras model yang sudah di-train .

Method :

Method	Deskripsi
<code>__init__(self, input_dim, units)</code>	Inisialisasi layer dengan dimensi input dan jumlah hidden units.
<code>load_weights(self, keras_layer)</code>	Memuat weight matrices (W_x , W_h , bias) dari Keras SimpleRNN layer.
<code>forward(self, x)</code>	Melakukan forward pass melalui sequence. Input shape $[\text{batch_size}, \text{seq_len}, \text{input_dim}]$, output shape $[\text{batch_size}, \text{units}]$. Iterasi melalui setiap timestep, update hidden state menggunakan persamaan $h_t = \tanh(x_t @ W_x + h_{t-1} @ W_h + b)$.

Contoh penggunaan: untuk batch 32 captions dengan panjang 35 tokens dan embed_dim 256, forward pass akan mengkonversi integer token sequence menjadi dense embeddings dengan shape $[32, 35, 256]$.

1.1.11. Kelas `LSTMLayer(src/rnn/layers/lstm.py)`

Merepresentasikan LSTM (Long Short-Term Memory) cell yang memperbaiki SimpleRNN dengan mekanisme gating untuk menangani long-range dependencies. Layer ini mempertahankan dual memory: cell state c_t untuk long-term memory dan hidden state h_t untuk short-term output. LSTM layer ini diimplementasikan dari scratch menggunakan NumPy dengan 4 gates (input, forget, output) yang di-compute dari weight matrices yang di-load dari Keras model.

Method :

Method	Deskripsi
<code>__init__(self, input_dim, units)</code>	Inisialisasi layer dengan dimensi input dan jumlah hidden units.
<code>load_weights(self, keras_layer)</code>	Memuat weight matrices untuk 4 gates (W_{xi} , W_{xf} , W_{xg} , W_{xo} , W_{hi} , W_{hf} , W_{hg} , W_{ho}) dan biases dari Keras LSTM layer.
<code>forward(self, x)</code>	Melakukan forward pass dengan gating mechanism. Input shape $[\text{batch_size}, \text{seq_len}, \text{input_dim}]$, output shape $[\text{batch_size}, \text{units}]$. Untuk setiap timestep: compute 4 gates (i_t , f_t , g_t , o_t), update cell

	state $c_t = f_t * c_{t1} + i_t * g_t$, compute hidden state $h_t = o_t * \tanh(c_t)$.
<code>_sigmoid(x)</code>	Helper method untuk menghitung sigmoid activation function.

Konteks penggunaan: LSTM digunakan sebagai decoder utama dalam image captioning task. Kemampuannya dalam mempertahankan informasi visual sepanjang caption generation membuatnya menghasilkan captions yang lebih koheren dan akurat dibandingkan SimpleRNN. Dual memory system memungkinkan model untuk selectively "remember" atau "forget" informasi sesuai kebutuhan.

1.1.12. Kelas `LSTMLayer(src/rnn/layers/lstm.py)`

Merepresentasikan fully-connected (dense) layer yang menerapkan linear transformation dengan bias dan optional activation function. Pada image captioning architecture, Dense layer digunakan dalam dua konteks: (1) projection layer untuk mengkonversi CNN feature vector ke embedding dimension (x_{-1}), dan (2) output layer untuk mengkonversi hidden state menjadi vocabulary logits untuk prediction.

Method :

Method	Deskripsi
<code>__init__(self, input_dim, output_dim, activation=None)</code>	Inisialisasi layer dengan dimensi input, dimensi output, dan jenis activation function.
<code>load_weights(self, keras_layer)</code>	Memuat weight matrix dan bias dari Keras Dense layer.
<code>forward(self, x)</code>	Melakukan forward pass: $output = x @ W + b$, kemudian aplikasikan activation function jika ada. Input shape [..., input_dim], output shape [..., output_dim].

Dalam architecture pre-inject, Dense projection layer mengubah CNN feature (2048-dim untuk InceptionV3) menjadi embedding_dim (256-dim), menghasilkan x_{-1} yang di-concatenate dengan caption embeddings. Dense output layer mengkonversi hidden state RNN/LSTM (256-dim) menjadi logits (vocab_size-dim) untuk menghasilkan probabilitas distribusi untuk prediksi token berikutnya.

1.1.13. Kelas CaptioningModelScratch(src/rnn/model.py)

Merepresentasikan complete image captioning pipeline yang menggabungkan encoder (pre-trained CNN) dan decoder (RNN/LSTM) dalam satu model. Class ini mengintegrasikan semua layer components dari scratch (embedding, recurrent, dense) untuk melakukan end-to-end inference: menerima image features dan menghasilkan caption sequence melalui greedy decoding

Method :

Method	Deskripsi
<code>__init__(self, keras_model, vocab, config)</code>	Inisialisasi model dengan pre-trained Keras model, vocabulary dict, dan konfigurasi (embed_dim, max_len, dll). Load weights dari Keras ke from-scratch layers.
<code>load_encoder_weights(self)</code>	Ekstrak dan load CNN feature dari pre-trained encoder (InceptionV3).
<code>load_decoder_weights(self)</code>	Ekstrak dan load RNN/LSTM decoder weights ke from-scratch layers.
<code>generate_caption(self, img_features, max_len)</code>	Genera satu caption untuk satu image. Implementasi greedy decoding: mulai dengan x_{-1} (projected CNN feature), iteratif predict next token dengan highest probability, stop saat token <end> atau max_len tercapai.
<code>generate_captions_batch(self, img_features_batch, max_len)</code>	Genera captions untuk batch images secara parallel.

Konteks penggunaan: CaptioningModelScratch adalah interface utama untuk inference dan evaluation. Ini menerima pre-computed image features dari CNN encoder dan menggunakan RNN/LSTM decoder untuk generate captions. Model ini digunakan untuk mengevaluasi performance dengan metrics seperti BLEU-4 dan METEOR, serta untuk melakukan qualitative analysis dengan membandingkan generated captions dengan ground truth captions.

Greedy decoding strategy dalam generate_caption memilih token dengan probabilitas tertinggi pada setiap step, berbeda dengan beam search yang mempertahankan multiple hypotheses. Strategi ini lebih cepat tetapi mungkin menghasilkan sub-optimal captions. Untuk evaluation, model juga dapat di-convert ke Keras inference mode untuk direct comparison antara from-scratch dan Keras implementations.

1.2. Penjelasan Forward Propagation

1.2.1. CNN

Forward propagation pada CNN dilakukan dengan meneruskan input melalui setiap layer secara berurutan hingga menghasilkan output prediksi. Proses ini dimulai dari layer pertama dan output dari suatu layer akan menjadi input bagi layer berikutnya. Setiap layer memiliki operasi yang berbeda sesuai fungsinya, seperti konvolusi, aktivasi, pooling, dan flattening.

Pada layer convolution seperti Conv2D, forward propagation dimulai dengan mengambil patch input sesuai ukuran kernel. Jika menggunakan padding "same", input terlebih dahulu ditambahkan padding nol pada bagian tepi agar ukuran output tetap sesuai. Kemudian, setiap patch input dikalikan dengan kernel yang bersesuaian dan hasilnya dijumlahkan untuk menghasilkan nilai feature map. Setelah itu, bias ditambahkan pada hasil konvolusi dan fungsi aktivasi diterapkan pada output tersebut.

Conv2D.forward
<pre>k = self.kernel.reshape(-1, C_out) # matrix multiply: [B, h_out*w_out, C_out] y = (col @ k) + self.bias y = y.reshape(B, h_out, w_out, C_out)</pre>

Proses konvolusi dilakukan untuk setiap channel input dan setiap filter output. Hasil akhir dari layer ini berupa feature map baru yang merepresentasikan pola tertentu dari input.

Pada layer LocallyConnected2D, proses forward propagation mirip dengan convolution, tetapi setiap posisi output memiliki bobot kernel yang berbeda. Fungsi mengambil patch input sesuai ukuran kernel, kemudian patch tersebut diubah menjadi vektor dan dikalikan dengan kernel pada posisi tersebut. Setelah ditambahkan bias, hasilnya diteruskan ke fungsi aktivasi.

LocallyConnected2D.forward
<pre>for i in range(H_out): for j in range(W_out): patch = x[:, i*sH:i*sH+kH, j*sW:j*sW+kW, :].reshape(B, -1) y[:, i, j, :] = patch @ self.kernel[pos] + self.bias[pos] pos += 1</pre>

Berbeda dengan convolution biasa yang menggunakan kernel yang sama untuk seluruh posisi, locally connected layer menggunakan kernel unik pada setiap lokasi output.

Pada layer pooling, forward propagation digunakan untuk melakukan downsampling feature map. Pada MaxPooling2D, setiap patch input akan diambil nilai maksimum, sedangkan pada AveragePooling2D akan dihitung nilai rata-ratanya.

MaxPooling2D.forward
<pre>windows = _pool_windows(x, self.pool_size, self.stride) ax = (2, 4) if self.stride == self.pool_size else (3, 4) return windows.max(axis=ax)</pre>

AveragePooling2D.forward
<pre>windows = _pool_windows(x, self.pool_size, self.stride) ax = (2, 4) if self.stride == self.pool_size else (3, 4) return windows.mean(axis=ax)</pre>

Selanjutnya, layer Flatten digunakan untuk mengubah feature map multidimensi menjadi vektor satu dimensi agar dapat digunakan oleh fully connected layer.

Flatten.forward
<pre>return x.reshape(x.shape[0], -1)</pre>

Pada setiap layer convolution maupun fully connected, output biasanya diteruskan ke fungsi aktivasi. Fungsi aktivasi seperti ReLU, Sigmoid, Tanh, dan Softmax digunakan untuk memberikan non-linearitas pada model. Sebagai contoh, fungsi ReLU akan mengubah semua nilai negatif menjadi 0.

Secara keseluruhan, forward propagation pada CNN dilakukan dengan meneruskan data secara bertahap melalui layer convolution atau locally connected untuk ekstraksi fitur, layer pooling untuk downsampling, layer flatten untuk perubahan bentuk data, dan fungsi aktivasi untuk menghasilkan representasi non-linear sebelum akhirnya menghasilkan output prediksi.

1.2.2. RNN

SimpleRNN adalah arsitektur recurrent neural network yang paling fundamental dalam memproses sequence data. Pada setiap timestep t , neuron RNN menerima input x_t dari data saat ini dan hidden state $h_{(t-1)}$ dari timestep

sebelumnya, kemudian menghitung hidden state baru h_t dengan melewati kombinasi linear dari kedua input tersebut melalui fungsi aktivasi tanh.

$$h_t = \tanh(x_t W_x + h_{(t-1)} W_h + b)$$

Persamaan di atas menunjukkan bahwa setiap hidden state bergantung pada dua komponen: (1) input baru x_t yang diproyeksikan melalui weight matrix W_x , dan (2) informasi dari timestep sebelumnya yang diproyeksikan melalui recurrent weight matrix W_h . Bias b ditambahkan untuk memberikan flexibility pada model. Fungsi aktivasi tanh dipilih karena menghasilkan nilai dalam rentang $[-1, 1]$, yang membantu dalam menjaga stabilitas gradient selama training.

Implementasi forward pass SimpleRNN melibatkan loop yang iteratif melalui setiap timestep dalam sequence. Berikut adalah pseudo-code untuk operasi tersebut:

```
h = zeros([batch_size, units]) # Initialize hidden state

for t in range(seq_len):
    x_t = X[:, t, :] # Input at timestep t
    z_t = dot(x_t, W_x) + dot(h, W_h) + b
    h = tanh(z_t) # Update hidden state

return h # Return final hidden state
```

Dalam implementasi batched, operasi matrix multiplication dilakukan secara paralel untuk seluruh sample dalam batch. Setiap timestep menerima input x_t dengan shape $[batch_size, input_dim]$ dan menggunakan hidden state h dari timestep sebelumnya dengan shape $[batch_size, units]$. Output akhir yang dikembalikan adalah hidden state terakhir $h_{(T-1)}$, yang merangkum informasi seluruh sequence dan digunakan sebagai representasi untuk tugas downstream seperti klasifikasi atau prediksi.

Meskipun SimpleRNN sederhana dan cepat, arsitektur ini memiliki keterbatasan signifikan dalam menangani sequence dengan dependensi jangka panjang. Masalah ini dikenal sebagai vanishing gradient problem. Ketika gradient di-backpropagate melalui banyak timestep (disebut backpropagation through time atau BPTT), nilai gradient dikalikan berkali-kali dengan jacobian dari fungsi aktivasi tanh, yaitu $(1 - h_t^2)$. Karena nilai ini selalu kurang dari 1 untuk kebanyakan hidden state, hasil perkalian gradient akan menurun secara

eksponensial seiring bertambahnya jumlah timestep. Hal ini menyebabkan model sangat sulit untuk belajar dan menyimpan dependensi dari timestep yang jauh di masa lalu.

$$dL/dh_t = (dL/dh_T) * prod_{k=t+1}^T (dh_k/dh_{(k-1)})$$

Akibat dari vanishing gradient problem ini, SimpleRNN biasanya hanya efektif untuk sequence dengan panjang sedang (sekitar 10-20 timestep). Untuk aplikasi yang memerlukan mempertahankan dependensi dalam sequence yang sangat panjang, seperti machine translation atau text generation yang membutuhkan konteks jangka panjang, SimpleRNN tidak cukup optimal dan perlu digantikan dengan arsitektur yang lebih sophisticated seperti LSTM.

1.2.3. LSTM

Long Short-Term Memory (LSTM) adalah variasi dari RNN yang dirancang khusus untuk mengatasi vanishing gradient problem dan memungkinkan model mempelajari dependensi jangka panjang dalam sequence. LSTM memperkenalkan mekanisme gating yang mengontrol aliran informasi melalui tiga gate: input gate, forget gate, dan output gate. Selain itu, LSTM juga mempertahankan cell state c_t yang berfungsi sebagai "memory belt" yang dapat menyimpan informasi dalam jangka panjang.

Pada setiap timestep t , LSTM melakukan operasi berikut. Pertama, model menghitung empat nilai gating dan candidate state:

$$i_t = \text{sigmoid}(x_t W_{xi} + h_{(t-1)} W_{hi} + b_i) \text{ [input gate]}$$

$$f_t = \text{sigmoid}(x_t W_{xf} + h_{(t-1)} W_{hf} + b_f) \text{ [forget gate]}$$

$$g_t = \tanh(x_t W_{xg} + h_{(t-1)} W_{hg} + b_g) \text{ [candidate state]}$$

$$o_t = \text{sigmoid}(x_t W_{xo} + h_{(t-1)} W_{ho} + b_o) \text{ [output gate]}$$

Forget gate f_t menghasilkan nilai antara 0 dan 1 (via sigmoid) yang mengontrol berapa banyak informasi dari cell state sebelumnya $c_{(t-1)}$ yang harus "dilupakan" atau dipertahankan. Input gate i_t mengontrol berapa banyak informasi baru dari candidate state g_t yang harus ditambahkan ke cell state. Candidate state g_t adalah calon state baru yang dikomputasi dari input saat ini dan hidden state sebelumnya, serupa dengan SimpleRNN tetapi menggunakan fungsi aktivasi tanh.

Setelah ketiga nilai ini dihitung, model melakukan update pada cell state:

$$c_t = f_t \cdot c_{(t-1)} + i_t \cdot g_t$$

Operasi di atas adalah kombinasi dari forget operation ($f_t \cdot c_{(t-1)}$) dan add operation ($i_t \cdot g_t$). Cell state yang baru adalah hasil dari scaling informasi lama dengan forget gate dan menambahkan informasi baru yang di-scale dengan input gate. Operasi $\cdot$ melambangkan element-wise multiplication. Kemudian, hidden state h_t dihitung dengan melewati cell state melalui tanh dan mengalikannya dengan output gate:

$$h_t = o_t \cdot \tanh(c_t)$$

Output gate o_t mengontrol informasi mana dari cell state yang harus di-expose sebagai hidden state output. Implementasi forward pass LSTM dalam bentuk pseudo-code adalah:

```

h = zeros([batch_size, units])
c = zeros([batch_size, units])

for t in range(seq_len):
    x_t = X[:, t, :]

    # Compute gates
    i_t = sigmoid(dot(x_t, W_xi) + dot(h, W_hi) + b_i)
    f_t = sigmoid(dot(x_t, W_xf) + dot(h, W_hf) + b_f)
    g_t = tanh(dot(x_t, W_xg) + dot(h, W_hg) + b_g)
    o_t = sigmoid(dot(x_t, W_xo) + dot(h, W_ho) + b_o)

    # Update cell state dan hidden state
    c = f_t * c + i_t * g_t
    h = o_t * tanh(c)

return h

```

Keunggulan utama LSTM dalam mengatasi vanishing gradient adalah bahwa cell state di-update melalui operasi addition (bukan multiplication), sehingga gradient dapat mengalir langsung melalui cell state tanpa perlu dikalikan dengan derivative dari fungsi aktivasi. Hal ini disebut constant error flow karena gradient dapat mempertahankan nilainya yang cukup besar saat di-backpropagate melalui banyak timestep:

$$dc_t/dc_{(t-1)} = f_t$$

Karena f_t adalah sigmoid output (nilai antara 0 dan 1), gradient tidak menghilang sepenuhnya dan dapat terus mengalir ke timestep yang lebih jauh. Ini memungkinkan LSTM untuk secara efektif belajar dependensi yang jauh, sehingga cocok untuk aplikasi dengan long-range dependencies seperti language modeling, machine translation, dan speech recognition.

Selain mengatasi vanishing gradient, LSTM juga memberikan kontrol yang lebih detail terhadap informasi dalam sequence. Forget gate memungkinkan model untuk melupakan informasi lama yang tidak relevan (ketika $f_t \approx 0$) atau mempertahankan informasi penting (ketika $f_t \approx 1$). Input gate memungkinkan selective addition dari informasi baru. Output gate memungkinkan model untuk memilih komponen apa dari cell state yang harus di-expose sebagai output. Fleksibilitas ini membuat LSTM jauh lebih powerful dibandingkan SimpleRNN dalam memodelkan kompleksitas dari sequential data.

2. Hasil Pengujian

2.1 CNN

2.1.1. Perbandingan Custom(Shared) dan Keras

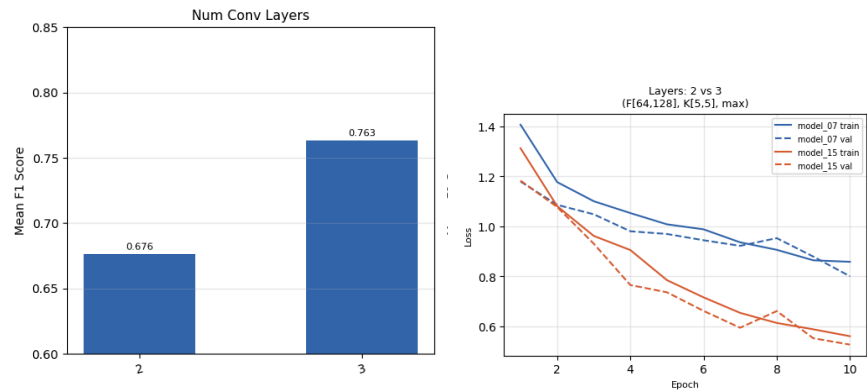
CLASSIFICATION REPORT - CUSTOM (SHARED)					
		precision	recall	f1-score	support
	buildings	0.70	0.88	0.78	437
	forest	0.96	0.95	0.96	474
	glacier	0.83	0.73	0.78	553
	mountain	0.78	0.80	0.79	525
	sea	0.78	0.88	0.83	510
	street	0.91	0.69	0.78	501
	accuracy			0.82	3000
	macro avg	0.83	0.82	0.82	3000
	weighted avg	0.83	0.82	0.82	3000

CLASSIFICATION REPORT - KERAS (SHARED)					
		precision	recall	f1-score	support
	buildings	0.70	0.88	0.78	437
	forest	0.96	0.95	0.96	474
	glacier	0.83	0.73	0.78	553
	mountain	0.78	0.80	0.79	525
	sea	0.78	0.88	0.83	510
	street	0.91	0.69	0.78	501
	accuracy			0.82	3000
	macro avg	0.83	0.82	0.82	3000
	weighted avg	0.83	0.82	0.82	3000

Berdasarkan gambar classification report dari model keras dan custom model dengan shared parameter, keduanya memiliki nilai macro f1 score sebesar 0.82. Hal tersebut berarti bahwa implementasi forward propagation untuk model CNN from scratch berhasil memproduksi hasil inferensi yang konsisten pada data uji. Tidak terdapat perbedaan performa pada metrik macro F1-score, sehingga implementasi model dapat dianggap sesuai dengan model Keras untuk proses forward propagation dengan shared parameter.

2.1.2. Perbandingan Hyperparameter

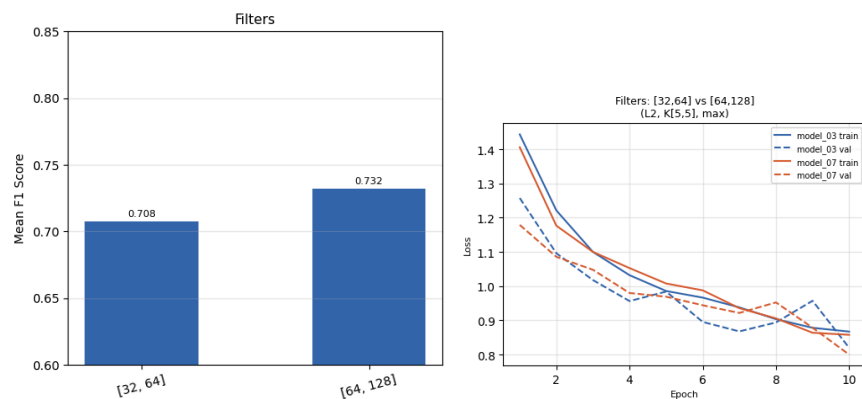
a. Jumlah Layer



Nilai macro F1-score menunjukkan bahwa model dengan 3 layer konvolusi memiliki performa yang lebih baik dibandingkan mode dengan 2 layer konvolusi. Terdapat perbedaan sekitar 0.087 pada rata-rata nilai macro F1-score ketika convolution layer ditambah dari 2 menjadi 3 layer.

Pada grafik loss terlihat bahwa model 3 layer menunjukkan penurunan training loss dan validation loss yang lebih cepat dan stabil sampai mencapai nilai yang lebih rendah dibanding model 2 layer. Model dengan 3 layer mampu mempelajari fitur yang lebih representatif dibandingkan dengan model dengan 2 layer. Penambahan layer konvolusi, CNN dapat mengekstraksi fitur yang lebih kompleks. Akibatnya kemampuan klasifikasi meningkat, terlihat dari nilai macro F1-score yang lebih tinggi dan validation loss yang lebih rendah.

b. Jumlah Filter

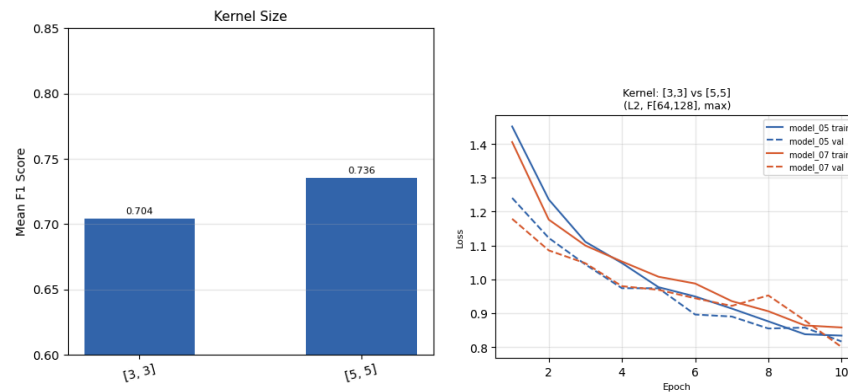


Nilai macro F1-score menunjukkan bahwa model dengan jumlah filter [64, 128] memiliki performa yang lebih baik dibandingkan mode dengan filter [32, 64]. Terdapat perbedaan sekitar 0.024 pada rata-rata nilai macro F1-score ketika jumlah filter diperbesar.

Pada grafik loss terlihat bahwa model dengan filter [64, 128] menunjukkan training loss dan validation loss yang cenderung lebih rendah pada epoch akhir dibanding

model dengan filter [32, 64]. Hal ini menunjukkan bahwa jumlah filter yang lebih besar membantu model menangkap lebih banyak pola fitur dari citra. Semakin banyak filter yang digunakan, semakin banyak variasi fitur yang dapat dipelajari pada setiap convolution layer. Akibatnya kemampuan klasifikasi meningkat, terlihat dari nilai macro F1-score yang lebih tinggi dan validation loss yang lebih rendah.

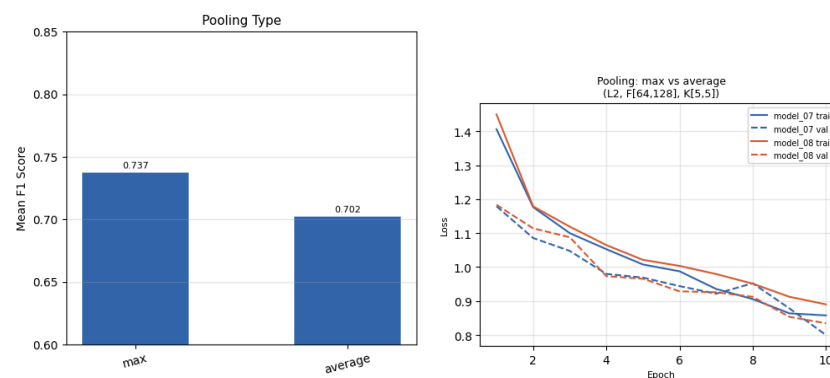
c. Ukuran Kernel



Nilai macro F1-score menunjukkan bahwa model dengan ukuran kernel [5,5] memiliki performa yang lebih baik dibandingkan model dengan ukuran kernel [3, 3]. Terdapat perbedaan sekitar 0.032 pada rata-rata nilai macro F1-score ketika ukuran kernel diperbesar.

Pada grafik loss terlihat bahwa model dengan kernel [5,5] menunjukkan training loss dan validation loss yang cenderung lebih stabil dan mencapai nilai akhir lebih rendah dibanding model dengan ukuran kernel [3, 3]. Hal ini menunjukkan bahwa ukuran kernel yang lebih besar membantu model menangkap pola global dan konteks spasial yang lebih luas sehingga fitur yang dipelajari lebih representatif.

d. Tipe Pooling



Nilai macro F1-score menunjukkan bahwa model dengan max pooling memiliki performa yang lebih baik dibandingkan model dengan average pooling. Terdapat

perbedaan sekitar 0.035 pada rata-rata nilai macro F1-score ketika max pooling digunakan.

Pada grafik loss terlihat bahwa model dengan max pooling menunjukkan training loss dan validation loss yang cenderung turun lebih cepat dan mencapai nilai akhir lebih rendah dibanding model dengan average pooling. Hal ini menunjukkan bahwa max pooling lebih efektif dalam mempertahankan fitur penting dibandingkan average pooling.

2.1.3. Perbandingan Shared dan Non Shared

CLASSIFICATION REPORT - CUSTOM (SHARED)						CLASSIFICATION REPORT - CUSTOM (NON SHARED)					
		precision	recall	f1-score	support			precision	recall	f1-score	support
	buildings	0.70	0.88	0.78	437		buildings	0.70	0.88	0.78	437
	forest	0.96	0.95	0.96	474		forest	0.96	0.95	0.96	474
	glacier	0.83	0.73	0.78	553		glacier	0.83	0.73	0.78	553
	mountain	0.78	0.80	0.79	525		mountain	0.78	0.80	0.79	525
	sea	0.78	0.88	0.83	510		sea	0.78	0.88	0.83	510
	street	0.91	0.69	0.78	501		street	0.91	0.69	0.78	501
	accuracy			0.82	3000		accuracy			0.82	3000
	macro avg	0.83	0.82	0.82	3000		macro avg	0.83	0.82	0.82	3000
	weighted avg	0.83	0.82	0.82	3000		weighted avg	0.83	0.82	0.82	3000

Berdasarkan hasil evaluasi model custom CNN shared dan custom CNN Non-shared parameter, keduanya memiliki nilai macro f1 score sebesar 0.82. Hal tersebut berarti bahwa kedua pendekatan menghasilkan performa klasifikasi yang setara pada data uji. Implementasi convolution baik menggunakan shared parameter maupun non-shared mampu mempelajari representasi fitur yang mirip pada dataset yang digunakan.

2.2 Simple RNN dan LSTM untuk Image Captioning

2.2.1. Perbandingan Jumlah Layer: RNN

Eksperimen dilakukan dengan memvariasikan jumlah layer recurrent pada decoder RNN dengan ukuran hidden state 128 dan 512

Konfigurasi	BLEU-4	METEOR	ms/img
rnn_L1_H128	0.0973	0.2896	5.6
rnn_L1_H512	0.1144	0.3245	9.5
rnn_L2_H128	0.0911	0.2911	5.8
rnn_L2_H512	0.1256	0.3130	10.5
rnn_L3_H128	0.1240	0.3101	5.7
rnn_L3_H512	0.0002	0.0982	15.4

Penambahan jumlah layer tidak selalu meningkatkan performa. Model rnn_L2_H512 mencapai BLEU-4 tertinggi (0.1256), sedangkan rnn_L3_H512 justru memiliki nilai BLEU-4 mendekati 0. Hal ini kemungkinan terjadi karena *vanishing gradient* pada konfigurasi hidden state yang cukup besar pada banyak layer tanpa regularisasi yang cukup. Model dengan 3 layer dan hidden 128 (rnn_L3_H128) masih dapat bersaing dengan BLEU-4 0.1240. Ukuran hidden yang lebih kecil dapat menekan terjadinya overfitting.

Untuk RNN pada eksperimen ini, 2 layer dengan hidden size 512 adalah konfigurasi paling optimal. Menambah layer di atas 2 dengan ukuran hidden layer yang besar berisiko menurunkan kinerja model.

2.2.2. Perbandingan Jumlah Layer: LSTM

Konfigurasi	BLEU-4	METEOR	ms/img
lstm_L1_H128	0.0003	0.1071	6.5
lstm_L1_H512	0.0002	0.1064	15.1
lstm_L2_H128	0.0001	0.1039	7.5
lstm_L2_H512	0.0003	0.1030	24.4
lstm_L3_H128	0.0004	0.1072	10.6
lstm_L3_H512	0.0004	0.1049	29.6

Seluruh varian LSTM from scratch menunjukkan BLEU-4 yang sangat rendah (< 0.001). Nilai METEOR sedikit lebih tinggi dibanding BLEU-4 yang menunjukkan model tetap menghasilkan beberapa kata yang relevan. Dari eksperimen ini, model terbaik LSTM adalah lstm_L3_H128 dengan BLEU-4 = 0.0004 dan METEOR = 0.1049.

Penambahan layer LSTM tidak memberikan dampak signifikan dalam kondisi ini karena performa keseluruhan sudah tidak baik sejak layer 1.

2.2.3. Perbandingan Ukuran Hidden State: RNN

Hidden Size	1 Layer	2 Layer	3 Layer
-------------	---------	---------	---------

128	0.0973	0.0911	0.1240
512	0.1144	0.1256	0.0002

Tabel BLEU-4 pada Variasi Hidden State RNN

Ukuran hidden 512 umumnya menghasilkan BLEU-4 lebih tinggi dibanding 128 untuk 1–2 layer. Namun untuk 3 layer, hidden 512 justru gagal dengan nilai BLEU-4 yang mendekati 0. Ini menunjukkan bahwa kapasitas model yang semakin besar dapat menyebabkan training yang tidak stabil dan tidak menjamin keakuratan. Hidden size yang lebih besar dapat meningkatkan performa pada jumlah layer tertentu, tetapi berisiko pada sebuah jaringan yang lebih dalam.

2.2.4. Perbandingan Ukuran Hidden State: LSTM

Hidden Size	Layer 1 BLEU-4	Layer 2 BLEU-4	Layer 3 BLEU-4
128	0.0003	0.0001	0.0004
512	0.0002	0.0003	0.0004

Tabel BLEU-4 pada Variasi Hidden State LSTM

Perbedaan antara hidden 128 dan 512 untuk LSTM from *scratch* sangat kecil menandakan keduanya sama-sama tidak bekerja dengan baik.

2.2.4 Perbandingan RNN vs LSTM

Konfigurasi	BLEU-4	METEOR	ms/img
rnn_L1_H128	0.0973	0.2896	5.6
rnn_L1_H512	0.1144	0.3245	9.5
rnn_L2_H128	0.0911	0.2911	5.8
rnn_L2_H512	0.1256	0.3130	10.5
rnn_L3_H128	0.1240	0.3101	5.7
rnn_L3_H512	0.0002	0.0982	15.4

Konfigurasi	BLEU-4	METEOR	ms/img
lstm_L1_H128	0.0003	0.1071	6.5
lstm_L1_H512	0.0002	0.1064	15.1
lstm_L2_H128	0.0001	0.1039	7.5
lstm_L2_H512	0.0003	0.1030	24.4
lstm_L3_H128	0.0004	0.1072	10.6
lstm_L3_H512	0.0004	0.1049	29.6

RNN from scratch jauh mengungguli LSTM from scratch pada semua metrik. BLEU-4 RNN terbaik 314 kali lebih tinggi dibanding LSTM terbaik. Waktu inferensi per gambar hampir identik (~10.5 ms vs 10.6 ms).

Secara umum, decoder RNN mampu menghasilkan kalimat yang bermakna dan relevan secara kontekstual, sedangkan LSTM from scratch cenderung mengulang token yang sama atau menghasilkan kalimat pendek yang tidak deskriptif, menunjukkan decoder gagal belajar bahasa dan perkataan.

2.2.5. Perbandingan Keras vs From Scratch

Konfigurasi	Implementasi	BLEU-4	METEOR	ms/img
rnn_L2_H512	Scratch	0.1256	0.3130	10.5
rnn_L2_H512	Keras	0.1256	0.3130	1791.7
lstm_L3_H128	Scratch	0.0004	0.1072	10.6
lstm_L3_H128	Keras	0.0004	0.1072	2006.9

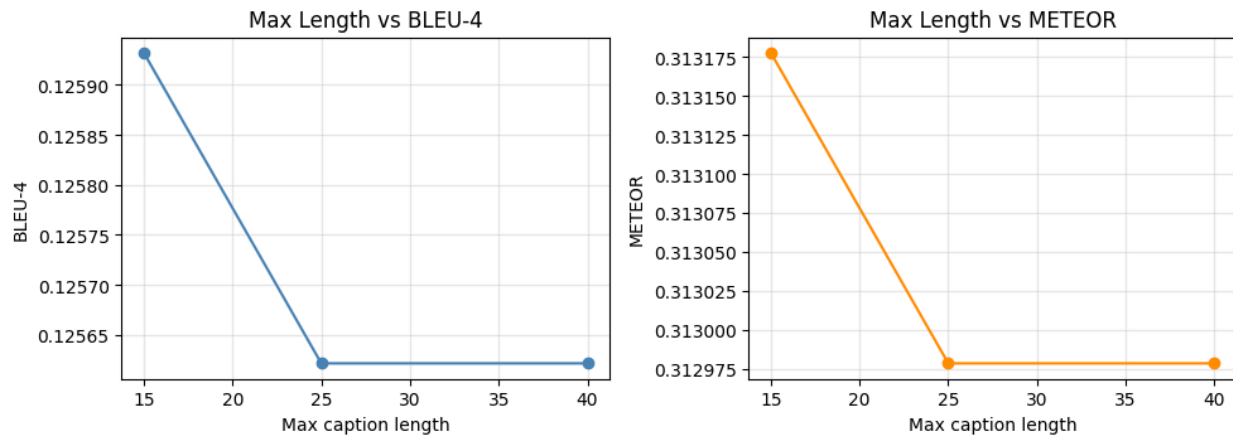
Skor serupa dapat dihasilkan oleh model Keras maupun *from scratch*. Namun, terdapat perbedaan kecepatan yang cukup signifikan. Hal ini menunjukkan bahwa implementasi *forward propagation from scratch* mampu mereproduksi hasil Keras dengan cukup baik. Perbedaan kecepatan dapat terjadi karena overhead yang cukup besar pada model Keras. Implementasi *from scratch* menggunakan Numpy yang membuat ukuran overhead lebih kecil dan perhitungan lebih cepat.

2.2.6. Pengaruh Panjang Maksimum Caption terhadap BLEU-4

Eksperimen dilakukan pada model terbaik secara keseluruhan (rnn_L2_H512 scratch) dengan tiga variasi max_len.

max_len	BLEU-4	METEOR	Rata-rata Panjang Caption
15	0.1259	0.3132	9.7 kata
25	0.1256	0.3130	9.7 kata
40	0.1256	0.3130	9.7 kata

Effect of Max Caption Length — rnn_L2_H512



Perubahan max_len dari 15 hingga 40 tidak mengubah rata-rata panjang caption yang dihasilkan (9.7 kata). Ini menunjukkan bahwa model secara konsisten menghasilkan token <end> sebelum mencapai batas maksimum, sehingga batas panjang yang lebih besar tidak berpengaruh pada output aktual. BLEU-4 pada max_len=15 sedikit lebih tinggi (0.1259 vs 0.1256).

Untuk dataset Flickr8k, max_len=15 sudah cukup memadai. Nilai max_len yang lebih besar tidak meningkatkan kualitas caption selama model sudah belajar menghasilkan token <end> pada waktu yang tepat. Maka, pengurangan max_len ke 12–15 dapat sedikit meningkatkan presisi BLEU-4.

3. Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan, implementasi CNN from scratch menghasilkan performa yang konsisten dengan model referensi Keras. Pada perbandingan antara model custom shared dan Keras shared, keduanya memperoleh nilai macro f1-score sebesar 0,82. Hal tersebut menunjukkan implementasi forward propagation telah berjalan dengan benar dan mampu mereproduksi hasil inferensi dari model Keras.

Pada perbandingan antara model custom shared dan custom non-shared juga menunjukkan nilai macro f1-score yang sama. Hal ini berarti bahwa penggunaan shared maupun non-shared pada implementasi CNN menghasilkan performa klasifikasi yang setara pada dataset yang digunakan. Dengan demikian, mekanisme weight sharing tidak memberikan perbedaan performa yang signifikan pada eksperimen ini, meskipun dalam hal komputasi model custom shared membutuhkan waktu dan komputasi yang jauh lebih rendah.

Dari sisi hyperparameter, seluruh variasi uji menunjukkan pengaruh terhadap performa model. Penambahan convolution layer dari 2 menjadi 3 memberikan peningkatan performa paling signifikan, ditunjukkan oleh kenaikan macro f1-score dan penurunan validation loss yang lebih stabil. Peningkatan jumlah filter juga meningkatkan kemampuan model dalam mengekstraksi fitur sehingga menghasilkan performa yang lebih baik. Selain itu, penggunaan kernel berukuran 5x5 memberikan hasil lebih baik dibandingkan kernel 3x3 karena mampu menangkap informasi spasial yang lebih luas. Pada tipe pooling, max pooling menghasilkan performa yang lebih baik dibandingkan average pooling karena lebih efektif mempertahankan fitur dominan pada feature map.

Pada bagian RNN dan LSTM untuk image captioning, implementasi decoder RNN from scratch berhasil menghasilkan BLEU-4 sebesar 0.1256 dan METEOR sebesar 0.3130, yang identik dengan implementasi Keras. Namun terdapat perbedaan kecepatan yang sangat signifikan. Implementasi from scratch menggunakan NumPy sekitar 170 kali lebih cepat dibandingkan Keras, yaitu sekitar 10.5 ms per gambar berbanding 1791.7 ms per gambar. Perbedaan ini dapat terjadi karena *overhead* pada Keras yang cukup besar untuk inferensi per gambar dibandingkan NumPy.

Sebaliknya, implementasi LSTM from scratch menghasilkan BLEU-4 yang sangat rendah (kurang dari 0.001) pada seluruh variasi konfigurasi, meskipun implementasi Keras untuk model LSTM yang sama juga menghasilkan skor yang sama rendahnya.

Perbandingan RNN dan LSTM menunjukkan bahwa dalam eksperimen ini, decoder RNN jauh mengungguli LSTM dalam hal kualitas *caption* yang dihasilkan. Analisis kualitatif pada 10 gambar dari test set Flickr8k menunjukkan bahwa RNN mampu menghasilkan kalimat yang bermakna dan relevan secara kontekstual, sementara LSTM from scratch cenderung menghasilkan kalimat repetitif atau tidak deskriptif. Secara teoritis, LSTM seharusnya lebih

unggul dengan kemampuan memori jangka panjangnya yang lebih kompleks, tapi dalam praktik ini keunggulan tersebut tidak dapat diamati.

Variasi jumlah layer dan ukuran hidden state pada RNN menunjukkan bahwa konfigurasi 2 layer dengan hidden size 512 merupakan titik optimal, dengan BLEU-4 tertinggi sebesar 0.1256. Penambahan layer di atas titik ini justru menyebabkan penurunan performa, terutama pada hidden size besar, akibat vanishing gradient yang tidak tertangani pada arsitektur SimpleRNN.

Eksperimen variasi panjang maksimum caption menunjukkan bahwa nilai `max_len` tidak berpengaruh signifikan terhadap BLEU-4 selama model sudah belajar menghasilkan token akhir pada waktu yang tepat. Model secara konsisten menghasilkan caption dengan rata-rata panjang 9.7 kata, jauh di bawah batas maksimum yang diuji (15, 25, dan 40 kata), sehingga peningkatan batas tidak memberi dampak pada kualitas output.

3.2. Saran

Berdasarkan kendala dan temuan yang diperoleh selama pengerjaan tugas ini, terdapat beberapa saran untuk pengembangan lebih lanjut.

Pertama, pada implementasi LSTM from scratch, perlu dilakukan verifikasi ulang terhadap cara implementasi dan loading bobot dari Keras. Selain itu, dapat dilakukan pengujian numerik tiap unit untuk setiap gate sebelum menjalankan evaluasi penuh, agar kesalahan dapat dideteksi lebih awal.

Kedua, untuk implementasi CNN, dapat digunakan regularisasi pada variasi dengan jumlah layer dan filter yang besar, guna mencegah overfitting yang dapat menurunkan performa pada data uji.

Ketiga, implementasi batch inference pada seluruh modul from scratch disarankan untuk dilakukan agar inferensi dapat dilakukan secara paralel, sehingga lebih efisien ketika digunakan pada dataset yang besar. Hal ini juga akan memperkecil gap kecepatan antara implementasi NumPy dan Keras pada skenario batch.

C. Pembagian Tugas

NIM	Nama	Tugas
13523025	Joel Hotlan Haris Siahaan	CNN: utility, forward propagation, pelatihan model, analisis dan evaluasi, laporan
13523030	Julius Arthur	Encoder, feature extraction untuk RNN/LSTM, Encoder Decoder pipeline, laporan

13523056	Salman Hanif	RNN/LSTM forward propagation, preprocessing caption, pelatihan decoder, analisis RNN vs LSTM, laporan
----------	--------------	---

D. Referensi

- Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.
- <https://numpy.org/doc/stable/index.html>
- https://edunexcontentprodhot.blob.core.windows.net/edunex/nullfile/1776050663153_IF3270-Mgg09-RNN-Part-1pptx?sv=2024-11-04&spr=https&st=2026-04-13T03%3A24%3A22Z&se=2028-04-12T03%3A24%3A22Z&sr=b&sp=r&sig=KaSH4um%2B3Q044GTU2Vwsjf%2BjjkivPCDZNMiDF8jsNZ8%3D&rsct=application%2Fpdf
- https://edunexcontentprodhot.blob.core.windows.net/edunex/2026/88964-Machine-Learning-Parent-Class/434708-Recurrent-Neural-Network/file/1776654833169_IF3270-Mgg1011-RNN-part2pptx?sv=2024-11-04&spr=https&st=2026-04-20T03%3A13%3A52Z&se=2028-04-19T03%3A13%3A52Z&sr=b&sp=r&sig=jTI6jTmva%2Beaj0gg6n3HRpt6AFf5XICD%2BNIJfHmx2yo%3D&rsct=application%2Fpdf

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : Joel Hotlan Haris Siahaan (13523025), Julius Arthur (13523030),
Salman Hanif (13523056)

Fakultas/Sekolah : STEI

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 1: Feedforward Neural Network

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak~~ menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/ Tidak *	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	
Lainnya, Jelaskan:		

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 15 Mei 2026



Joel Hotlan Haris Siahaan
13523025



Julius Arthur
13523030



Salman Hanif
13523056

